

**КІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ТАРАСА ШЕВЧЕНКА**

Факультет комп'ютерних наук та кібернетики
Кафедра інтелектуальних програмних систем

Курсова робота

за спеціальністю 121 Інженерія програмного забезпечення
на тему:

**РОЗРОБКА ТА ТЕСТУВАННЯ ВИСОКОНАВАНТАЖЕНОГО REST
API СИСТЕМИ БРОНЮВАННЯ ЗАЛІЗНИЧНИХ КВИТКІВ**

Виконав студент 3-го курсу
ОПП “Програмна інженерія”
Тесленко Назар Олександрович

(підпис)

Науковий керівник: асистент кафедри
інтелектуальних програмних систем,
доктор філософії
Ількун Олександр Петрович

(підпис)

Засвідчую, що в цій роботі немає
запозичень з праць інших авторів без
відповідних посилань.

Студент

(підпис)

КИЇВ 2026

РЕФЕРАТ

Обсяг роботи: 34 сторінки, 4 таблиці, 1 рисунок, 12 джерел посилання, 2 додатки.

CONCURRENCY CONTROL, REST API, POSTGRESQL, TYPESCRIPT, FASTIFY, ТЕСТУВАННЯ ПРОДУКТИВНОСТІ, БРОНЮВАННЯ КВИТКІВ, НАВАНТАЖУВАЛЬНЕ ТЕСТУВАННЯ, ЦІЛІСНІСТЬ ДАНИХ.

Об'єктом роботи є процес забезпечення цілісності даних при конкурентному доступі у високонавантажених вебзастосунках. Предметом роботи виступає серверна частина системи бронювання залізничних квитків, реалізована за вимогами REST API.

Метою курсової роботи є розробка та дослідження продуктивності високонавантаженого REST API на основі порівняльного аналізу патернів керування конкурентним доступом із забезпеченням цілісності даних при пікових навантаженнях.

Використані методи розробки: патерни керування конкурентним доступом, проєктування реляційних баз даних, навантажувальне тестування, порівняльний аналіз продуктивності. Використані інструменти розроблення: TypeScript, Node.js, Fastify, PostgreSQL, k6, React.

Результат роботи: спроектовано та реалізовано REST API системи бронювання залізничних квитків, досліджено та впроваджено можливі патерни керування конкурентним доступом — Naive, DB Constraint, Pessimistic Locking, Optimistic Locking, створені штучні умови навантажувального тестування кожного з патернів при кількості одночасних запитів в проміжку від 5 до 1000, розроблено аналітичний дашборд для візуалізації та порівняння отриманих метрик.

Робота була виконана з урахуванням сучасних підходів розробки вебзастосунків та методологій тестування продуктивності програмного забезпечення.

Застосування програмного продукту може бути як основа для побудови високонавантажених систем із суворими вимогами до цілісності даних. Отримані результати порівняльного аналізу патернів можуть слугувати практичними рекомендаціями у процесі проєктування та розробки архітектурних рішень для подібних систем.

ЗМІСТ

Скорочення та умовні позначення	6
Вступ.....	7
1 Аналіз предметної області та існуючих рішень	10
1.1 Проблема конкурентного доступу у вебзастосунках	10
1.2 Огляд існуючих підходів до керування конкурентним доступом	11
1.2.1 Популярні підходи вирішення конкурентного доступу	11
1.3 Огляд інструментів навантажувального тестування	12
2 Проєктування системи	14
2.1 Архітектура REST API.....	14
2.2 Архітектура тестового середовища.....	15
2.2.1 Генератор навантаження.....	15
2.2.2 Сервер застосунку.....	15
2.2.3 Сервер бази даних	16
2.2.4 Взаємодія між компонентами.....	16
2.3 Вибір та обґрунтування технологічного стеку	16
2.3.1 Мова програмування та середовище виконання	16
2.3.2 Вебфреймворк	17
2.3.3 База даних	17
2.3.4 Інструменти тестування	17
3 Реалізація патернів конкурентного доступу	19
3.1 Загальна архітектура серверної частини	19
3.2 Структура бази даних.....	20
3.3 Реалізація патернів конкурентного доступу	20
3.3.1 Патерн Naive.....	20
3.3.2 Патерн DB Constraint.....	21
3.3.3 Патерн Pessimistic Locking.....	21
3.3.4 Патерн Optimistic Locking.....	22
4 Тестування та аналіз результатів.....	23
4.1 Методологія навантажувального тестування	23
4.2 Умови проведення тестування.....	23
4.3 Результати навантажувального тестування	24
4.4 Тестування стабільності під тривалим навантаженням	26

4.5 Порівняльний аналіз патернів	27
Висновки.....	30
Перелік джерел посилання	31
Додаток А Графіки метрик навантажувального тестування.....	33
Додаток Б Графіки метрик Soak-тестування	34

СКОРОЧЕННЯ ТА УМОВНІ ПОЗНАЧЕННЯ

БД — база даних;

API — Application Programming Interface, інтерфейс прикладного програмування;

Avg — Average, середнє значення;

DB — Database, база даних;

HTTP — HyperText Transfer Protocol, протокол для передачі гіпертексту;

JSON — JavaScript Object Notation, текстовий формат обміну даними;

JWT — JSON Web Token, токен для передачі даних автентифікації

Latency — затримка, час від відправлення запиту до отримання відповіді від серверу;

P90 — 90-th Percentile, 90-й персентиль часу відповіді;

P95 — 95-th Percentile, 95-й персентиль часу відповіді;

REST — Representational State Transfer, архітектурний стиль взаємодії компонентів розподіленого застосунку в мережі.

RPS — Requests Per Second, кількість запитів за секунду;

SQL — Structured Query Language, мова структурованих запитів для БД;

Throughput — пропускна здатність системи, кількість успішно оброблених запитів за певну одиницю часу.

VU — Virtual User, віртуальний користувач.

ВСТУП

Оцінка сучасного стану об'єкта дослідження. Стрімкий розвиток цифрових сервісів та зростання кількості їх користувачів паралельно підвищують вимоги до надійності та продуктивності серверних застосунків. Системи обслуговування великої кількості одночасних запитів — зокрема платформи електронної комерції, системи бронювання та фінансові сервіси — мають справу з проблемами конкурентного доступу до спільних ресурсів. Некоректна обробка подібних ситуацій призводить до порушення цілісності даних, що є критичною проблемою для будь-якої виробничої системи.

У сьогоденні існує ряд апробованих підходів до вирішення даної проблеми на рівні БД: песимістичне та оптимістичне блокування, використання обмежень цілісності, а також багатокрокові резервування. Попри існування цих рішень, порівняльний аналіз продуктивності при пікових навантаженнях систем залишається актуальною задачею, оскільки вибір патернів безпосередньо впливає як на коректність роботи системи, так і на її пропускну здатність та затримку відповіді.

Актуальність роботи та підстави для її виконання. Забезпечення цілісності даних при конкурентному доступі є однією з фундаментальних потреб розробки серверної частини програмного забезпечення. Некоректний вибір механізму обробки, при сотнях або тисячах запитів до одного ресурсу, може призвести як до логічних помилок — наприклад, продажу одного місця кільком користувачам — так і до суттєвого падіння продуктивності системи в цілому. Актуальність роботи зумовлена необхідністю практичного дослідження та кількісного порівняння існуючих патернів в умовах екстремальних навантажень, що дозволить обґрунтовано підходити до вибору архітектурних рішень при проєктування подібних систем.

Мета й завдання роботи. Метою курсової роботи є розробка та дослідження продуктивності високонавантаженого REST API на основі порівняльного аналізу патернів керування конкурентним доступом із забезпеченням цілісності даних при пікових навантаженнях.

Визначені задачі для виконання поставленої мети:

- Проаналізувати існуючі підходи керування конкурентним доступом у реляційних базах даних;
- Спроекувати схему бази даних та архітектуру REST API для системи бронювання залізничних квитків;
- Реалізувати чотири патерни керування конкурентним доступом: Naive, DB Constraint, Pessimistic Locking та Optimistic Locking;
- Розробити сценарії навантажувального тестування із емуляцією від 5 до 1000 одночасних запитів на один ресурс;
- Зібрати та порівняти метрики продуктивності кожного патерну: коректність даних, Latency(avg, p90, p95, max) та Throughput;
- Розробити аналітичний дашборд для візуалізації результатів тестування.

Об'єкт, методи й засоби розроблення. Об'єктом роботи є процес забезпечення цілісності даних при конкурентному доступі у високонавантажених вебзастосунках. Предметом роботи виступає серверна частина системи бронювання залізничних квитків, реалізована за вимогами REST API.

Використані методи. Патерни керування конкурентним доступом, проєктування реляційних баз даних, навантажувальне тестування, порівняльний аналіз продуктивності. Використані інструменти розроблення:

- TypeScript — мова програмування, використана як основна. Забезпечує безпеку, щодо типізації об'єктів та інших структур, що підвищує надійність та підтримуваність коду;

- Node.js – середовище виконання JavaScript на стороні сервера. Побудоване на рушії V8;
- Fastify – високопродуктивний вебфреймворк для Node.js, оптимізований для роботи з великою кількістю запитів.
- PostgreSQL – об’єктно-реляційна система управління базами даних з підтримкою.
- k6 – open-source інструмент навантажувального тестування, що дозволяє симулювати велику кількість віртуальних користувачів.
- React – бібліотека для побудов користувацьких інтерфейсів.

Можливі застосування. Результати роботи можуть бути застосовані при проєктуванні та розробці високонавантажених систем із суворими вимогами до цілісності даних — зокрема систем бронювання, електронної комерції, фінансових платформ та інших сервісів де критично важливим є коректна обробка конкурентних запитів на спільні ресурси.

1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ІСНУЮЧИХ РІШЕНЬ

1.1 Проблема конкурентного доступу у вебзастосунках

Популярні вебзастосунки надають послуги тисячам своїх користувачів одночасно, що неминуче породжує ситуації коли декілька запитів намагаються отримати доступ до одного і того ж ресурсу в один момент часу. Така ситуація називається конкурентним доступом і є однією з фундаментальних проблем розробки серверної частини програмного забезпечення.

Як класичний приклад розглянемо систему бронювання квитків: якщо двоє користувачів одночасно переглядають одне й те саме вільне місце та майже одночасно роблять дію купівлі, виникає типова ситуація *race condition* — стан гонитви. У випадках відсутності механізму урегулювання обидва запити можуть обробитися з успішним статусом, що приводить до продажу одного місця двом різним користувачам, і відповідно викликає порушення цілісності даних.

З теоретичної точки зору стан гонитви (*race condition*) виникає в системах з паралельним доступом тоді, коли кінцевий результат виконання залежить від відносного порядку або часу виконання кількох паралельних операцій. У контексті реляційних баз даних ця проблема описується як «втрачене оновлення», або ж *Lost Update*, що проявляється між операціями читання та запису: перша транзакція читає стан умовного ресурсу та визначає його як доступний, після чого інша транзакція встигає змінити цей стан до того, як перший запит завершує запис [1].

Наслідки некоректної обробки конкурентного доступу поділяються на два типи. Перший — звичайні логічні помилки цілісності даних, такі як подвійний продаж ресурсу, некоректні залишки або дублювання записів. Другий — наслідкові проблеми продуктивності. Спроби запобігти подвійним продажам за допомогою жорсткого блокування можуть призводити до штучних затримок у

відповідях сервера, зменшення загальної пропускну здатності або ж виникнення взаємних блокувань транзакцій, що може призводити до зупинки обробки запитів.

Вирішення проблеми конкурентного доступу вимагає застосування певних алгоритмів керування, кожен з яких по-різному балансує між гарантіями цілісності даних та продуктивності системи. Вибір одного з патернів залежить від специфіки системи, а саме її характеристик навантаження, вимог до цілісності та допустимого рівня затримки відповіді.

1.2 Огляд існуючих підходів до керування конкурентним доступом

У реляційних базах даних існує ряд усталених підходів до вирішення проблем race conditions. Кожен з них реалізує відмінну стратегію синхронізації і має власний набір характеристик з точки зору коректності та продуктивності.

1.2.1 Популярні підходи вирішення конкурентного доступу

— Максимальний рівень ізоляції транзакцій — Serializable

Є найвищим рівнем ізоляції за стандартами SQL. При використанні даного підходу реляційна база даних автоматично гарантує, що паралельне виконання транзакцій призведе до такого ж результату, якби вона виконувалася строго послідовно. Ціною такого безпекового заходу є зниження пропускну здатності та часті відхилення транзакцій з боку бази даних при конфліктах, що потребує реалізації логіки повторних спроб на стороні сервера [3].

— Песимістичне блокування — Pessimistic Locking

Підхід, що базується на припущенні про високу ймовірність виникнення конфліктів. У цьому випадку ресурс жорстко блокується на рівні бази даних (наприклад, використання SQL-конструкції «SELECT ... FOR UPDATE») ще на етапі його читання [2], [4]. Поки перша транзакція не завершиться, інші транзакції, що намагаються отримати доступ до цього ж ресурсу, очікують у черзі. Але в таких випадках можливе виникнення взаємних блокувань, що тягне за собою зниження продуктивності серверу та оцінок інших метрик [5].

— Оптимістичне блокування — Optimistic Locking

Підхід, що базується на припущенні рідкості конфліктів. Замість блокування ресурсу під час читання, кожному запису в базі даних додається спеціальне поле версії (або мітки часу). Під час збереження змін система перевіряє, чи не змінилася версія ресурсу іншою транзакцією з моменту початкового читання [4]. В іншому випадку — транзакція відхиляється. Недоліком цього методу є перекладення обов’язків обробки помилок на API, яке має обробити помилку або ініціювати алгоритм повторної спроби.

Таблиця 1.1 — Порівняльний аналіз стратегій конкурентного доступу

Критерій	Serializable	Песимістичне блокування	Оптимістичне блокування
Пропускна здатність	Низька	Середня	Висока
Рівень відхилення транзакцій	Високий	Середній	Відсутній
Складність реалізації (рівень БД)	Низька	Середня	Середня
Складність реалізації (рівень API)	Висока	Низька	Висока
Оптимальний сценарій використання	Фінансові системи	Процеси з частими конфліктами	Системи з високим навантаженням

Аналізуючи вище розглянуті підходи, можемо зрозуміти, що не існує універсального рішення. Вибір між оптимістичним та песимістичним блокуванням безпосередньо залежить від профілю навантаження системи.

1.3 Огляд інструментів навантажувального тестування

Для проведення навантажувального тестування існує ряд спеціалізованих інструментів, кожен з яких має власні особливості щодо підходу до генерації навантаження, мови написання сценаріїв та можливостей аналізу.

— Apache JMeter – один з найбільш поширених інструментів з відкритим вихідним кодом. Має графічний інтерфейс для створення сценаріїв тестування та підтримує широкий спектр протоколів. Написаний на Java, що зумовлює відносно високе споживання ресурсів при великій кількості VU.

— Gatling – інструмент навантажувального тестування орієнтований на високу продуктивність. Сценарії пишуться мовою Scala з використанням DSL. Генерує детальні HTML-звіти з графіками розподілу затримок.

— Locust – інструмент створений на Python. Дозволяє описувати сценарії тестування засобами Python-коду, підтримує моніторинг в реальному часі та розподілене тестування.

— k6 – сучасний інструмент навантажувального тестування розроблений компанією Grafana Labs. Створений на Go, проте має опис сценарії мовою JavaScript. Підтримує експорт результатів у різних форматах та інтеграцію з системами моніторингу.

Таблиця 1.2 — Порівняльний аналіз інструментів навантажувального тестування

Критерій	JMeter	Gatling	Locust	k6
Мова сценаріїв	XML/Groovy	Scala	Python	JavaScript
Графічний інтерфейс	Так	Ні	Так	Ні
Споживання ресурсів	Високе	Низьке	Середнє	Низьке
Експорт результатів	JSON, CSV	HTML	CSV	JSON, CSV

2 ПРОЄКТУВАННЯ СИСТЕМИ

2.1 Архітектура REST API

При проєктуванні серверної частини системи бронювання залізничних квитків ключовим завданням був вибір архітектурних рішень, що забезпечують як коректну обробку конкурентних запитів, так і можливість об'єктивного порівняння різних патернів керування конкурентним доступом в однакових умовах.

Для побудови програмного інтерфейсу обрано архітектурний стиль REST. Завдяки стандартним HTTP-методам структура запитів і відповідей стає прозорою та передбачуваною. Це значно полегшує як розробку клієнтської частини, так і написання скриптів для навантажувального тестування. Відсутність стану на стороні сервера між запитами дозволяє коректно симулювати незалежні конкурентні запити від різних користувачів під час тестування [6].

Серверна частина організована за модульним принципом з розділенням відповідальності між трьома шарами. Шар маршрутизації відповідає за реєстрацію ендпоінтів та визначення HTTP-методів. Шар контролерів обробляє вхідні запити та формує відповіді. Шар сервісів містить бізнес-логіку та взаємодію з базою даних. Така організація відповідає принципу єдиної відповідальності та забезпечує слабку зв'язаність між компонентами, що спрощує незалежну розробку та тестування кожного модуля.

Принциповим архітектурним рішенням є виділення окремих ендпоінтів для кожного патерну керування конкурентним доступом замість єдиного універсального. Це рішення зумовлене головною метою роботи — порівняльним аналізом патернів. Ізольовані ендпоінти дозволяють проводити навантажувальне тестування кожного патерну незалежно, виключаючи взаємний вплив між ними та забезпечуючи чисті результати вимірювань.

Для зберігання даних обрано реляційну модель. Реляційні бази даних надають вбудовані механізми керування транзакціями та блокуваннями, що є необхідною умовою для реалізації досліджуваних патернів конкурентного доступу.

2.2 Архітектура тестового середовища

Для проведення об'єктивного дослідження патернів конкурентного доступу було спроектовано архітектуру тестового середовища що складається з трьох ізольованих компонентів: генератора навантаження, сервера застосунку та сервера бази даних. Така організація дозволяє імітувати реальні умови експлуатації високонавантаженого сервісу та чітко розмежувати споживання системних ресурсів між компонентами під час тестування.

2.2.1 Генератор навантаження

Генератор навантаження симулює поведінку великої кількості одночасних користувачів шляхом формування щільного потоку HTTP-запитів до програмного інтерфейсу системи.

Для уникнення штучних обмежень операційної системи — зокрема вичерпання пулу динамічних портів та затримок пов'язаних із закриттям TCP-з'єднань — запуск генератора здійснюється в середовищі з ядром Linux. Це гарантує що вузьким місцем під час тестування є саме логіка застосунку або база даних, а не обмеження клієнтського середовища.

2.2.2 Сервер застосунку

Сервер застосунку є центральним компонентом системи що приймає вхідний потік запитів та виступає оркестратором створюваних транзакцій. Завдяки асинхронній архітектурі сервер здатен обробляти велику кількість паралельних з'єднань без блокування потоку виконання. В контексті операції бронювання сервер діє за наступним алгоритмом:

— приймає запит,

- ініціює транзакцію в базі даних,
- застосовує обраний патерн керування конкурентним доступом,
- повертає клієнту результат — успішне бронювання або відповідну помилку конфлікту.

2.2.3 Сервер бази даних

Сервер бази даних є компонентом на рівні якого фізично виникають стани гонитви та застосовуються механізми їх вирішення. База даних відповідає за управління транзакціями, дотримання вимог ACID та накладання блокувань на рівні окремих рядків [9].

2.2.4 Взаємодія між компонентами

Взаємодія між компонентами відбувається синхронно по ланцюжку:

- генератор навантаження формує запит,
- сервер застосунку його обробляє та звертається до бази даних,
- результат повертається у зворотному напрямку.

Така модель взаємодії дозволяє інструменту тестування точно фіксувати ключові метрики продуктивності.

2.3 Вибір та обґрунтування технологічного стеку

2.3.1 Мова програмування та середовище виконання

Як основну мову розробки серверної частини було обрано TypeScript. Статична типізація дозволяє виявляти помилки на етапі компіляції, що має значення при роботі з транзакціями та логікою синхронізації. Оскільки обрана мова є надбудовою над JavaScript, маємо повну сумісність з Node.js та широкий вибір бібліотек [10].

Node.js було обрано як середовище виконання серверної частини. Асинхронна модель виконання забезпечує ефективну обробку великої кількості одночасних з'єднань без блокування потоку виконання, що є критичним для перевірки витримки пікових навантажень [11].

2.3.2 Вебфреймворк

Fastify обрано як HTTP-фреймворк замість більш поширеного Express завдяки суттєво вищій продуктивності. За даними офіційних бенчмарків Fastify обробляє близько 46000 запитів на секунду, що приблизно в чотири рази перевищує показники Express [7]. Для системи орієнтованої на дослідження продуктивності ця різниця є принциповою — вона дозволяє мінімізувати вплив фреймворку на результати тестування патернів та отримати більш чисті метрики.

2.3.3 База даних

PostgreSQL обрано як систему керування базами даних завдяки розвиненій підтримці механізмів керування конкурентним доступом. Підтримка операцій блокування записів, обмежень унікальності та перелічуваних типів даних є вбудованими можливостями PostgreSQL, що безпосередньо використовуються при реалізації досліджуваних патернів [8].

Для взаємодії з базою даних використовується бібліотека postgres з написанням запитів на чистому SQL без використання ORM. Такий підхід забезпечує повний контроль над запитами що виконуються — зокрема над конструкціями блокування та керування транзакціями.

2.3.4 Інструменти тестування

k6 обрано як інструмент навантажувального тестування з обґрунтувань наведених у підрозділі 1.2.2.

Окрім раніше наведених характеристик k6, інструмент розроблений мовою програмування Go і використовує горутини (Goroutines) — легковагові потоки виконання, що керуються безпосередньо планувальником середовища виконання мови Go. Це дозволяє генерувати величезний потік запитів при мінімальних витратах оперативної пам'яті та процесорного часу, що є критично важливим для отримання достовірних результатів без спотворення метрик [12].

Сценарії тестування написані на JavaScript, що є природним для проєкту на TypeScript та Node.js. Результати кожного тесту експортуються у форматі JSON та передаються на аналітичний дашборд через відповідний ендпоінт API.

3 РЕАЛІЗАЦІЯ ПАТЕРНІВ КОНКУРЕНТНОГО ДОСТУПУ

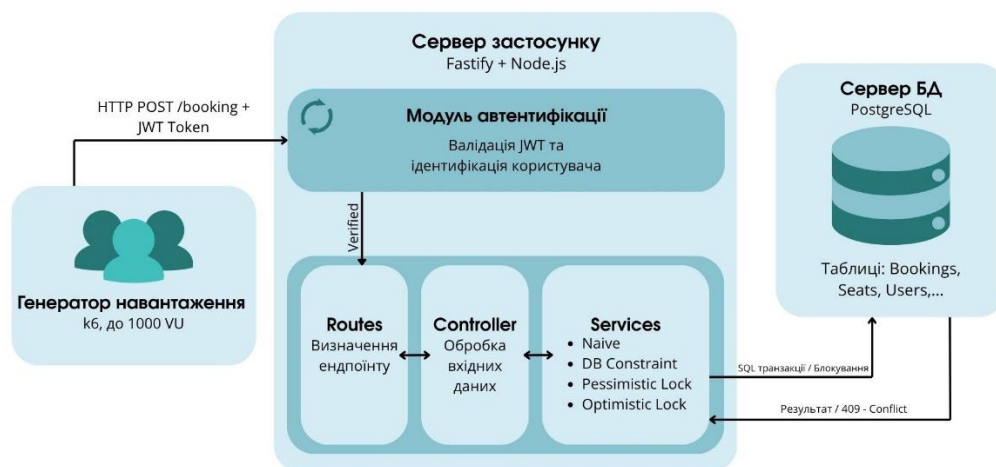
3.1 Загальна архітектура серверної частини

Серверна частина системи реалізована з використанням фреймворку Fastify та мови програмування TypeScript. Код організовано за модульним принципом де кожен функціональний модуль відповідає за окрему предметну область системи. Виділено п'ять основних модулів: автентифікація, станції, маршрути, місця та бронювання. Кожен модуль складається з трьох компонентів наведених — файл маршрутизації, контролера та сервісу — що відповідають архітектурі трьох рівнів описаній у підрозділі 2.1.

Основним, з точки зору предмету дослідження, є модуль бронювань. На відміну від інших модулів, що реалізують стандартні операції читання та запису, модуль бронювань містить чотири окремих методи сервісного шару для кожного досліджуваного патерну. Кожному методу відповідає окремий ендпоінт що дозволяє проводити ізольоване навантажувальне тестування кожного патерну незалежно від інших.

Детальну структурну схему взаємодії модулів серверної частини, включаючи проходження запиту через шар автентифікації та обробку відповідним патерном конкурентного доступу, наведено на рисунку 3.1.

Рисунок 3.1 — Загальна архітектура серверної частини та потік обробки запитів



Автентифікація користувачів реалізована на основі JWT. Усі ендпоінти модуля бронювань є захищеними — перед виконанням операції сервер перевіряє наявність та коректність токена і визначає ідентифікатор користувача, що ініціював запит.

3.2 Структура бази даних

База даних містить вісім таблиць, що відображають основні сутності предметної області. Таблиця *users* зберігає облікові дані користувачів системи. Таблиці *stations* та *routes* описують станції та маршрути між ними. Таблиці *trains* та *wagons* описують потяги та їх вагонний склад. Таблиця *seats* містить перелік місць у кожному вагоні. Таблиця *trips* поєднує потяг, маршрут, та конкретну дату і час відправлення.

Таблиця *bookings* є головним об'єктом спостереження. Вона містить обмеження унікальності `UNIQUE(trip_id, seat_id)`, що на рівні бази даних унеможливило б подвійний продаж одного місця на один рейс. Поле *version* у таблиці *seats* використовується патерном Optimistic Locking та збільшується при кожній зміні запису.

Для патерну Naïve реалізована окрема таблиця *bookings_naive*, яка навмисно не містить обмежень унікальності. Такий підхід дозволить наглядно продемонструвати наслідки повної відсутності захист від конкурентного доступу в чистому вигляді.

3.3 Реалізація патернів конкурентного доступу

3.3.1 Патерн Naïve

Патерн Naïve реалізує операцію бронювання без будь-яких механізмів захисту від конкурентного доступу і слугує первинною точкою для порівняльного аналізу. Алгоритм складається з двох послідовних операцій: читання поточного стану місця та запису нового бронювання. Якщо в цей же

момент інший конкурентний запит зчитує той самий рядок до того, як перший встиг виконати оновлення, обидва запити ідентифікують місце як доступне.

Оскільки в початковому алгоритмі не застосовані механізми блокування, СУБД не обмежує доступ інших транзакцій. У результаті, обидва запити виконують операцію бронювання, вважаючи, що вони отримали право на це місце, що призводить до подальших логічних помилок у системі. Фактично, відсутність ізоляції транзакцій дозволяє конкурентним запитам перезаписувати результати один одного створюючи конфлікт при отриманні ресурсу, причому повертаючи успішні статуси.

3.3.2 Патерн DB Constraint

Патерн DB Constraint реалізує рівень захисту цілісності даних виключно засобами СУБД без додаткової логіки на рівні сервера. Єдиним механізмом захисту є обмеження унікальності `UNIQUE(trip_id, seat_id)` визначене на рівні схеми таблиці *bookings*.

Алгоритм зводиться до виконання єдиної операції `INSERT`. У разі спроби вставити запис що порушує обмеження унікальності PostgreSQL автоматично відхиляє транзакцію [8]. Сервер перехоплює відповідну помилку бази даних та повертає клієнту відповідь з кодом 409 — `Conflict`. Перевагою підходу є мінімальний вплив на продуктивність за відсутності явних блокувань.

3.3.3 Патерн Pessimistic Locking

Патерн Pessimistic Locking реалізує стратегію жорсткого блокування ресурсу на рівні бази даних, ґрунтуючись на припущенні про високу ймовірність конфліктів при паралельному доступі.

Алгоритм виконується в рамках єдиної транзакції бази даних. На першому кроці виконується запит із конструкцією `SELECT FOR UPDATE`, що встановлює виключне блокування на рядок (row-level lock), що відповідає обраному місцю [2]. Всі інші транзакції, що намагаються отримати доступ до цього ж рядка

переходять у стан очікування до завершення поточної транзакції. На другому етапі перевіряється відсутність існуючого бронювання для даного місця за даним маршрутом. За умови відсутності конфлікту виконується запис нового бронювання та транзакція завершується.

При виникненні взаємних блокувань або перевищенні часу очікування сервер обробляє відповідні помилки бази даних як 409 — Conflict. Цей підхід забезпечує гарантовану цілісність даних, проте при високій інтенсивності запитів може призводити до зростання затримок через велику конкуренцію між потоками.

3.3.4 Патерн Optimistic Locking

Патерн Optimistic Locking відрізняється від Pessimistic Locking, оскільки реалізує стратегію без явного блокування ресурсів, що ґрунтується на припущенні про низьку ймовірність конфліктів. Замість блокування рядка під час читання система відстежує зміни через поле *version* у таблиці *seats*.

Алгоритм складається з трьох кроків, що виконуються в рамках транзакції: На першому кроці зчитується поточне значення поля *version* для обраного місця. Другим етапом, виконується спроба оновлення — операція UPDATE інкрементує значення *version* лише за умови, що воно залишилось незмінним з моменту читання. Якщо інша транзакція встигла змінити стан місця раніше, умова не виконується і операція повертає нуль змінених рядків, що є сигналом конфлікту. На фінальному етапі, за відсутності конфлікту виконується запис нового бронювання, інакше — сервер повертає відповідь із статусом 409. Перевагою підходу є відсутність черги очікування: всі запити виконуються паралельно, що значно підвищує пропускну здатність системи за низької конкуренції.

4 ТЕСТУВАННЯ ТА АНАЛІЗ РЕЗУЛЬТАТІВ

4.1 Методологія навантажувального тестування

Навантажувальне тестування проводилось з використанням інструменту k6 за сценарієм симуляції конкурентного доступу до одного спільного ресурсу. Основним сценарієм є одночасна спроба великої кількості користувачів забронювати одне і те саме місце на одному рейсі. Такий сценарій є найбільш показовим для виявлення відмінностей між патернами оскільки максимізує ймовірність виникнення конфліктів та наочно демонструє поведінку кожного механізму захисту в умовах пікового навантаження.

Для кожного патерну запускався окремий тест з однаковими параметрами навантаження. Кількість ітерацій дорівнювала кількості віртуальних користувачів — тобто кожен VU виконував рівно одну спробу бронювання. Перед кожним тестом стан бази даних скидався до початкового — видалялись всі бронювання для тестового місця та скидалось значення поля *version*. Це гарантує однакові початкові умови для кожного патерну та кожного рівня навантаження.

В рамках тестування збирались наступні метрики:

- коректність даних – кількість успішних бронювань для одного місця. Єдиним коректним результатом є рівно одне успішне бронювання незалежно від кількості одночасних запитів.

- Latency – час відповіді сервера. Фіксувались avg, p90, p95 та max.

- Throughput – кількість оброблених запитів за секунду (RPS).

4.2 Умови проведення тестування

Тестування проводилось в локальному середовищі з використанням операційної системи Windows 10 з підсистемою WSL2 (Windows Subsystem for Linux) на базі ядра Linux. Інструмент k6 запускався безпосередньо в середовищі WSL2, що дозволило уникнути обмежень мережевого стека Windows при

генерації великої кількості одночасних TCP-з'єднань. Серверна частина системи та PostgreSQL запускались локально.

Слід зазначити, що локальне середовище тестування має певні особливості порівняно з виробничим середовищем: генератор навантаження, сервер застосунку та база даних розміщені на одній фізичній машині що виключає мережеві затримки між компонентами. Отримані результати відображають відносні характеристики продуктивності патернів між собою, а не абсолютні показники що були б отримані в розподіленому середовищі.

Тестування проводилось для шести рівнів навантаження: 5, 10, 50, 100, 500 та 1000 віртуальних користувачів. Для кожного патерну та кожного рівня навантаження виконувався окремий тест з паузою між запусками для відновлення стабільного стану системи.

4.3 Результати навантажувального тестування

Для проведення навантажувального тестування за стратегією Spike-тестування використовувався виконавець per-vu-iterations інструменту k6, що забезпечує виконання рівно однієї ітерації кожним віртуальним користувачем. Оскільки сценарій тестування не містить штучних затримок, усі віртуальні користувачі ініціюють запити практично одночасно. Такий підхід гарантує, що загальна кількість паралельних запитів точно відповідає кількості віртуальних користувачів у момент часу, та максимізує конкуренцію за єдиний ресурс, унеможливаючи повторне виконання запитів одним VU.

Було проведене тестування для чотирьох реалізованих патернів: Naive, DB Constraint, Pessimistic Locking та Optimistic Locking при шести рівнях навантаження описаних у підрозділі 4.2. Результати тестування для рівня навантаження 1000 віртуальних користувачів, як найбільш показового, наведено в таблиці 4.1, детальний порівняльний аналіз для всіх рівнів навантаження у вигляді графіків представлено на рисунках А.1–А.3(Додаток А).

Таблиця 4.1 — Результати навантажувального тестування при 1000 віртуальних користувачів

1000 VU	Коректність	Avg,	P90,	P95,	Max,	RPS
Патерни		мс	мс	мс	мс	
Naive	16	223	438	474	530	115
DB Constraint	1/1	136	331	357	445	130
Pessimistic Lock	1/1	354	820	864	1274	122
Optimistic Lock	1/1	178	305	318	891	131

Аналіз графіків залежності середньої затримки від рівня навантаження (рисунки А.1) виявляє характерну поведінку кожного патерну.

DB Constraint демонструє найнижчу та найстабільнішу затримку в усьому діапазоні навантаження.

Optimistic Locking показує помірне зростання затримки.

Pessimistic Locking демонструє стабільне зростання затримки із збільшенням навантаження, що є прямим наслідком накопичення черги очікування.

Патерн Naive при рівні 50 VU демонструє аномальний пік затримки, після якого спостерігається часткове відновлення. Така нестабільна поведінка є типовою для алгоритмів без механізмів блокування: час обробки стає непередбачуваним через хаотичне виникнення стану гонитви та неконтрольовані спроби бази даних одночасно записати однакові дані.

Графік 95-го персентиля часу відповіді (p95) (рисунки А.2) відображає максимальний час очікування для 95% користувачів, відкидаючи поодинокі мережеві аномалії. Аналіз метрики додатково підкріплює результати середньої затримки: неблокуючі патерни Optimistic Locking (318 мс) та DB Constraint (357 мс) залишаються найстабільнішими при 1000 VU, успішно уникаючи довгих черг.

Натомість Pessimistic Locking, демонструє різке зростання p95 до 864 мс, що наочно підтверджує проблему накопичення транзакцій.

На графіку пропускної здатності (рисунок А.3) спостерігається загальна тенденція до зростання RPS із збільшенням кількості VU для всіх патернів. DB Constraint та Optimistic Locking демонструють найвищі показники пропускної здатності що підтверджує перевагу підходів без явних блокувань при високому навантаженні.

4.4 Тестування стабільності під тривалим навантаженням

З метою перевірки стабільності системи при тривалому рівномірному навантаженні було проведено Soak-тестування з використанням виконавця constant-vus інструменту k6. На відміну від пікового навантажувального тестування, де кожен VU виконує одну ітерацію, Soak-тест передбачає безперервне циклічне виконання запитів протягом визначеного часу. Це дозволяє виявити деградацію продуктивності, вичерпання пулу з'єднань або нестабільну поведінку під тривалим навантаженням. Тестування проводилось для 6 рівнів навантаження, зазначених у підрозділі 4.2, з тривалістю 30 секунд.

Результати Soak-тестування представлено на рисунках Б.1-Б.3(додаток Б)

Аналіз отриманих метрик (Таблиця 4.2) підтверджує та закріплює висновки, зроблені за результатами пікового навантажувального тестування.

Таблиця 4.2 – Результати Soak-тестування при 1000 віртуальних користувачах

1000 VU	Коректність	Avg,	P90,	P95,	Max,	RPS
Патерни		мс	мс	мс	мс	
Naive	18	449	770	851	1347	488
DB Constraint	1/1	75	188	262	811	642
Pessimistic Lock	1/1	354	524	596	911	506
Optimistic Lock	1/1	225	347	402	740	575

Усі три захищені патерни (DB Constraint, Optimistic Lock та Pessimistic Lock) довели свою надійність, забезпечивши сувору цілісність даних – рівно одне успішне бронювання. Відсутність блокувань у патерні Naïve призвела до запису 18 хибних дублікатів при тривалому навантаженні.

Ієрархія продуктивності також залишилася незмінною: патерн DB Constraint зберіг лідерство у швидкодії, тоді як Pessimistic Locking очікувано продемонстрував найвищі затримки серед захищених підходів.

Водночас абсолютні показники Soak-тестування суттєво відрізняються від результатів Spike-тесту: спостерігається значне зростання пропускну здатності (до 642 RPS для DB Constraint) та падіння середньої затримки (до 75 мс). Ця розбіжність пояснюється характером розподілу запитів. Під час Spike-тесту 1000 користувачів звертаються до сервера майже одночасно, миттєво переповнюючи пул з'єднань бази даних і створюючи величезну чергу очікування, що різко збільшує Latency. Натомість Soak-тест генерує рівномірне циклічне навантаження, дозволяючи базі даних миттєво обробляти запит і повертати з'єднання в пул для наступного користувача.

4.5 Порівняльний аналіз патернів

Аналіз сукупності отриманих результатів дозволяє сформулювати комплексну оцінку кожного досліджуваного патерну.

Патерн Naïve підтвердив очікувану поведінку – при рівні навантаження 1000 VU зафіксовано 16 успішних бронювань одного місця, що є прямим підтвердженням race condition. Слід зазначити, що результат патерну Naïve є недетермінованим – кількість порушень цілісності даних варіюється між запусками залежно від точного часового розподілу запитів. При низьких рівнях навантаження патерн може демонструвати коректний результат, що є наслідком випадкового порядку виконання операцій, а не наявності захисного механізму.

Патерн DB Constraint продемонстрував найкраще співвідношення продуктивності та коректності серед усіх досліджуваних підходів. Середня затримка 136 мс при RPS 130 є найкращим показником серед конкретних патернів при 1000 VU. Висока продуктивність пояснюється відсутністю явних блокувань – конфлікт вирішується на рівні індексу бази даних, що є найефективнішим з точки зору накладних витрат механізмом. Результати Soak-тестування підтвердили стабільність патерну при тривалому навантаженні.

Патерн Pessimistic Locking продемонстрував найвищу затримку серед інших патернів — середня затримка 354 мс при максимальній 1274 мс для 1000 VU. Така поведінка є прямим наслідком архітектури підходу: із збільшенням кількості одночасних запитів формується черга очікування, що суттєво збільшує час відповіді. Попри низьку продуктивність, патерн гарантує абсолютну коректність даних. Варто зазначити, що Pessimistic Locking має суттєве обмеження при прямому використанні: утримання блокування протягом усього часу транзакції унеможливорює його застосування в сценаріях, де між читанням і записом існує тривалий проміжок часу. Проте, саме логіка песимістичного блокування виступає надійним технічним фундаментом для реалізації патерну Soft Reservation (м'якого резервування). За такого підходу механізм жорсткого блокування використовується лише на коротку мить для фіксації проміжного статусу резервації ресурсу за конкретним користувачем, після чого транзакція бази даних завершується, вивільняючи ресурси та дозволяючи системі безпечно виконувати тривалу проміжну бізнес-логіку.

Патерн Optimistic Locking продемонстрував збалансовані характеристики – середня затримка 178 мс при RPS 131 для 1000 VU. Відсутність явних блокувань дозволяє всім запитам виконуватись паралельно, що забезпечує суттєво вищий Throughput порівняно з Pessimistic Locking. Запити, що зазнали конфлікту отримують відмову миттєво без очікування у черзі.

Узагальнені результати порівняльного аналізу свідчать про відсутність універсального рішення – кожен патерн демонструє певний компроміс між продуктивністю та складністю реалізації.

— DB Constraint забезпечує найвищу продуктивність при повній коректності даних, проте не дозволяє реалізувати складнішу бізнес-логіку перевірки перед записом.

— Optimistic Locking є оптимальним вибором для систем з помірним рівнем конкурентних звернень, де важлива висока пропускна здатність.

— Pessimistic Locking є доцільним для систем з найвищими вимогами до надійності та необхідністю виконання складної багатоетапної бізнес-логіки в процесі отримання ресурсів.

ВИСНОВКИ

У курсовій роботі було розроблено та досліджено продуктивність високонавантаженого REST API системи бронювання залізничних квитків з фокусом на порівняльному аналізі патернів керування конкурентним доступом.

В ході виконання роботи було вирішено такі завдання:

- Проаналізовано існуючі підходи до керування конкурентним доступом у реляційних базах даних та інструменти навантажувального тестування;

- Спроектовано архітектуру REST API та схему бази даних системи бронювання залізничних квитків з урахуванням вимог до реалізації досліджуваних патернів;

- Реалізовано чотири патерни керування конкурентним доступом: Naive, DB Constraint, Pessimistic Locking та Optimistic Locking;

- Розроблено сценарії навантажувального тестування із симуляцією від 5 до 1000 одночасних запитів на один ресурс з використанням інструменту k6;

- Проведено навантажувальне та Soak тестування кожного патерну з отриманням метрик коректності даних, Latency, Throughput;

- Розроблено аналітичний дашборд для візуалізації та порівняння результатів тестування.

Практична значимість роботи полягає в тому, що отримані результати порівняльного аналізу можуть слугувати практичними рекомендаціями при виборі патерну керування конкурентним доступом відповідно до вимог конкретної системи. Розроблений програмний продукт може бути використаний, як основа для побудови високонавантажених систем із суворими вимогами до цілісності даних.

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ

1. L. Cambi. The Lost Update Problem in Concurrency Control. Baeldung on Computer Science. URL: <https://www.baeldung.com/cs/concurrency-control-lost-update-problem> (дата звернення: 05.03.2025).
2. L. Albe. SELECT FOR UPDATE Considered Harmful in PostgreSQL. Cybertec. URL: <https://www.cybertec-postgresql.com/en/select-for-update-considered-harmful-postgresql/> (дата звернення: 05.03.2025).
3. Concurrency Control. Databricks Blog. URL: <https://www.databricks.com/blog/concurrency-control> (дата звернення: 07.03.2025).
4. Optimistic vs Pessimistic Locking. URL: <https://piresfernando.com/blog/optimistic-vs-pessimistic-lock> (дата звернення: 07.03.2025).
5. Pessimistic Locking vs Optimistic Locking. Modern Treasury. URL: <https://www.moderntreasury.com/learn/pessimistic-locking-vs-optimistic-locking> (дата звернення: 10.03.2025).
6. P. Powell. What Is a REST API (RESTful API)? IBM. URL: <https://www.ibm.com/think/topics/rest-apis> (дата звернення: 12.03.2025).
7. Fastify Benchmarks. URL: <https://fastify.dev/benchmarks> (дата звернення: 15.03.2025).
8. PostgreSQL Documentation. Explicit Locking. URL: <https://www.postgresql.org/docs/current/explicit-locking.html> (дата звернення: 15.03.2025).
9. PostgreSQL Documentation. Transaction Isolation. URL: <https://www.postgresql.org/docs/current/transaction-iso.html> (дата звернення: 15.03.2025).
10. TypeScript Documentation. URL: <https://www.typescriptlang.org/docs> (дата звернення: 20.03.2025).

11. Node.js Documentation. URL: <https://nodejs.org/en/docs> (дата звернення: 20.03.2025).

12. k6 Documentation. Executors. URL: <https://grafana.com/docs/k6/latest/using-k6/scenarios/executors> (дата звернення: 25.03.2025).

ДОДАТОК А

ГРАФІКИ МЕТРИК НАВАНТАЖУВАЛЬНОГО ТЕСТУВАННЯ

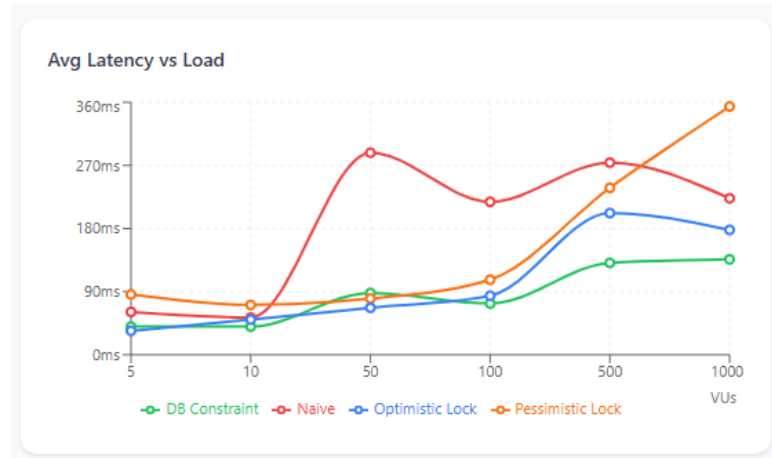


Рисунок А.1 – Графік залежності середньої затримки від рівня навантаження

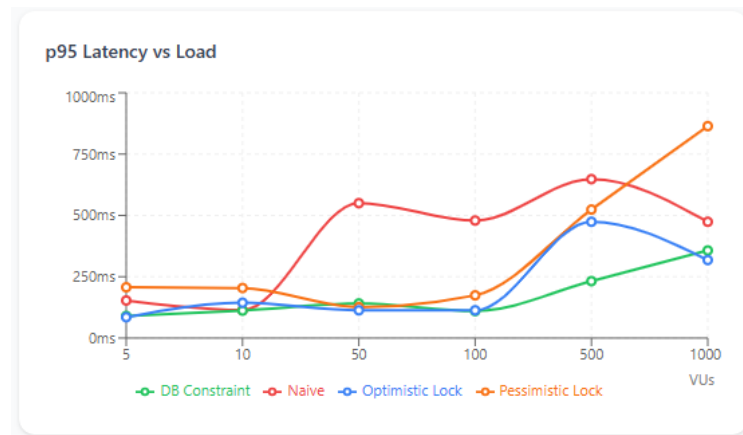


Рисунок А.2 – Графік залежності 95-го перцентилу часу відповіді (p95) від рівня навантаження

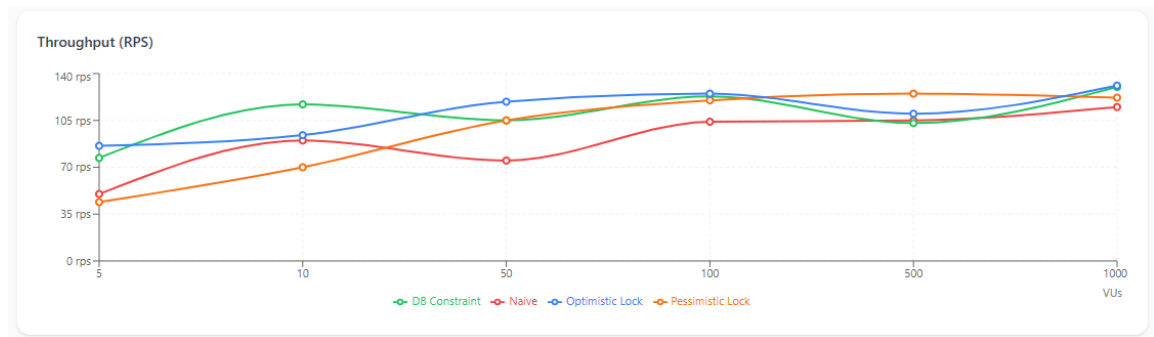


Рисунок А.3 – Графік пропускної здатності від рівня навантаження

ДОДАТОК Б

ГРАФІКИ МЕТРИК SOAK-ТЕСТУВАННЯ

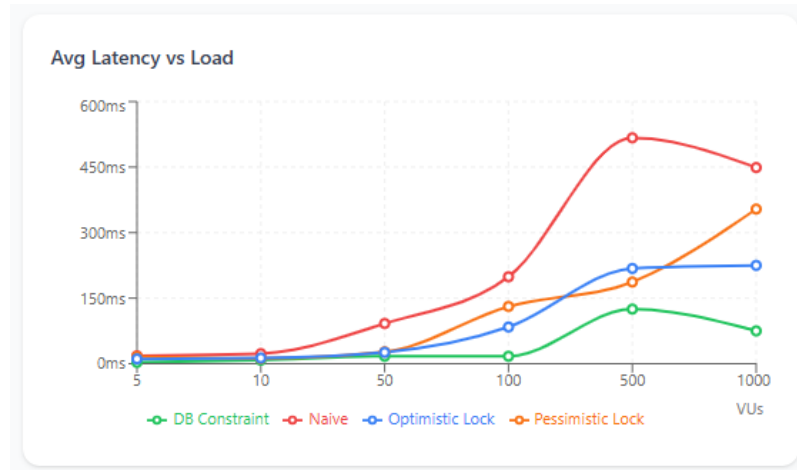


Рисунок Б.1 – Графік залежності середньої затримки від рівня навантаження

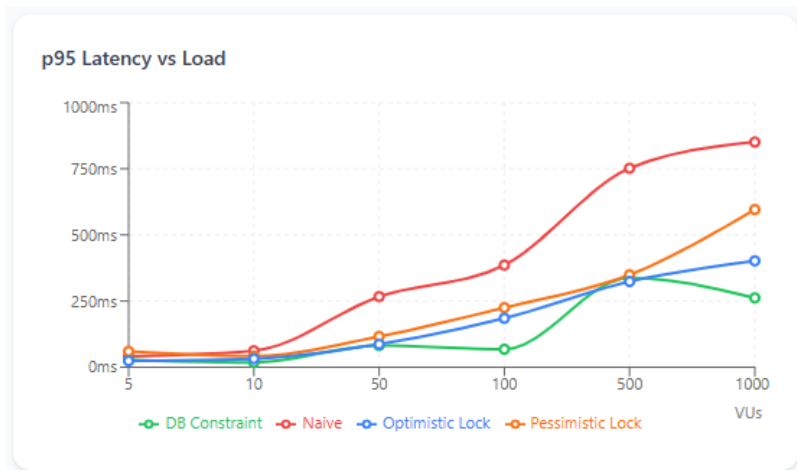


Рисунок Б.2 – Графік залежності 95-го персентилля часу відповіді (p95) від рівня навантаження

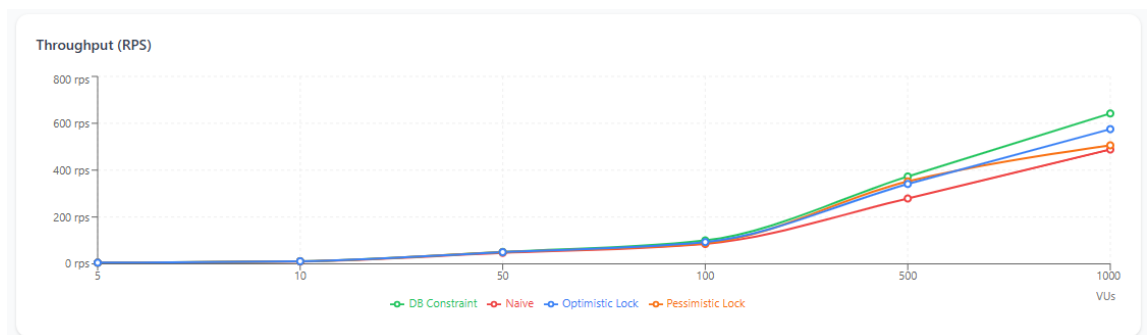


Рисунок Б.3 – Графік пропускної здатності від рівня навантаження